EmPath Implementation Roadmap

Purpose: build a Python prototype that compares residential access to temples, groceries and pharmacies using distance-only and environmental-friction routes.

Scope: Plano ZIP areas 75025, 75024, 75023 and 75074, represented by Census ZCTA boundaries. Budget: one week, 15 hours of core work plus up to 3 hours for troubleshooting and polish.

Minimum demonstration: approximately 5–10 verified residential origins per ZIP, individual destinations in three categories, one environmental scenario and one reused or adapted GeoAI component. Keep the four-area scope; reduce sample size or optional features if needed.

Workflow at a Glance

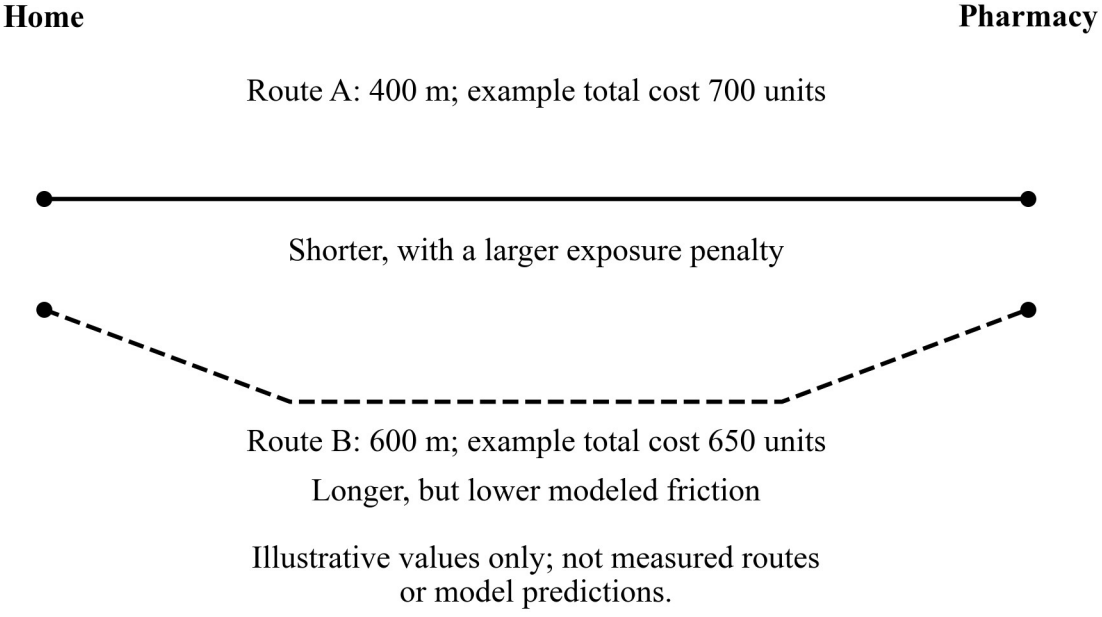
Step	Time	Deliverable
1. Inventory	0.5 h	Sources and coverage checklist
2. Clean inputs	2 h	Clean data and quality report
3. Build network	2.5 h	Walking graph and connection audit
4. Add features	2 h	Edge features and data dictionary
5. Adapt GeoAI	3 h	Model adapter and model card
6. Route and rank	1.5 h	Routes and residential rankings
7. Evaluate and map	2 h	Comparison results and interactive map
8. Package	1.5 h	README, slides and backup

How the Algorithms Work

Think of EmPath as a walking map with a burden score attached to each small path segment. The system uses six separate operations:

- 1. Build the map. A graph is a collection of dots and connecting lines. Dots represent junctions or access points; lines represent walkable segments. Homes and destinations attach to this network.
- 2. Describe each segment. Record distance and supported environmental information, such as a canopy estimate. A missing connection remains a gap; it is not repaired merely because two dots are close.
- 3. Use the existing AI model. The model estimates the quantity it was designed to predict, such as an environmental feature. A behavioral model may instead estimate shade–distance preferences. Neither model can infer every walking need from a map alone.
- 4. Assign walking costs. Convert supported estimates into a nonnegative segment cost, alongside distance. A cost is a comparison score, not automatically minutes or medical risk. Document any manually chosen penalty.
- 5. Choose a route. Dijkstra's algorithm tracks the lowest accumulated cost found while exploring connected paths. It returns a minimum-cost route; it does not need to list every possible route. The same algorithm can optimize distance or friction simply by changing the costs.
- 6. Compare homes. For each home, find the lowest-cost reachable temple, grocery and pharmacy. Add those three category costs with equal weights for the prototype. Lower totals rank better; flag missing services rather than quietly ignoring them.

Example: the shorter route below has a larger exposure penalty. The illustrative totals are 400 + 300 = 700 for Route A and 600 + 50 = 650 for Route B. Distance-only routing selects A; friction routing selects B. Real penalties require evidence or an explicit scenario assumption.



Where each tool fits: OSMnx obtains the map; GeoPandas and Rasterio prepare spatial features; the selected existing model estimates its target; NetworkX finds paths; pandas combines category costs; Folium displays the explanation. Routing and ranking are not themselves the AI model.

1 Inventory Sources and Coverage

- Open EmPath.code-workspace in VS Code. Select .venv/bin/python and the EmPath Jupyter kernel. Read README.md and docs/CONFIRMED_SCOPE.md.
- List boundaries, walking infrastructure, residential origins, destinations, environmental features and model inputs. For each ZIP, mark coverage as downloaded, partial, available online or unverified.
- Record provider, source URL, vintage, resolution, format, reuse terms, intended use and local file. Reuse scripts/audit_walking_coverage.py for walking coverage.

Tools: pandas, GeoPandas, requests. **Save:** sources table and four-area coverage checklist. **Done when** every required input has a status and next action.

2 Download and Clean Essential Inputs

- Run .venv/bin/python scripts/clean_data.py --zipcodes 75025 75024 75023 75074. Preserve raw snapshots; inspect the generated quality report.
- Check IDs, geometry, duplicates and missing values. Use EPSG:26914 for metric operations and EPSG:4326 for map display.
- Use Plano address points for a small verified residential sample. Parcels support checking but do not prove residential use or pedestrian entrances.
- Query OSM destination candidates: pharmacies, supermarkets/groceries and appropriate places of worship. Verify names, categories and access points. Missing tags do not prove that a facility is absent.

Tools: GeoPandas, Shapely, pandas, requests, OSMnx. **Save:** cleaned GeoPackages, origins.gpkg, destinations.gpkg and quality_report.md. **Done when** all four areas have documented coverage or explicit gaps.

3 Build and Audit the Pedestrian Network

- Use OSMnx graph_from_polygon with network_type="walk" and retain_all=True. Start with the four-ZCTA union plus roughly 500 m surrounding context; expand if routes are truncated.
- Save the original GraphML. Compare with Plano sidewalk surfaces and trails. Sidewalk polygons are supporting data, not ready-made routing edges.
- Connect origins and destinations through plausible access points. Audit nearest-node connectors for fences, inaccessible entrances and major crossings. Record components and disconnected origins.
- Inspect at least one route per ZIP. Add municipal paths only after checking access, topology and duplicates; preserve unreachable results.

Tools: OSMnx, NetworkX, GeoPandas, Folium. **Reference:** Street Shadow for graph-to-route structure; UNA for origin/destination architecture. **Save:** walking_graph.graphml, access_connectors.gpkg and network_audit.md.

4 Add Environmental Features

- Acquire a manageable clip of the Texas 2021 10 m canopy raster. Sample a documented corridor around each edge; start with a 10 m buffer and record that choice.
- Save edge ID, length_m, canopy_fraction, valid_pixel_fraction, source vintage and verified crossing constraints. Check ranges and missingness by ZIP; preserve NoData.
- Treat canopy as an environmental proxy, not time-specific shade or measured heat stress. Keep Tree Equity Score values as neighborhood context. Add weather, traffic, imperviousness or shadows only after core features work.

Tools: Rasterio, NumPy, GeoPandas, Shapely. **Reference:** CoolWalks for shade-sensitive costs. **Save:** edge_features.gpkg and DATA_DICTIONARY.md. **Done when** every scoring feature has units, provenance and coverage checks.

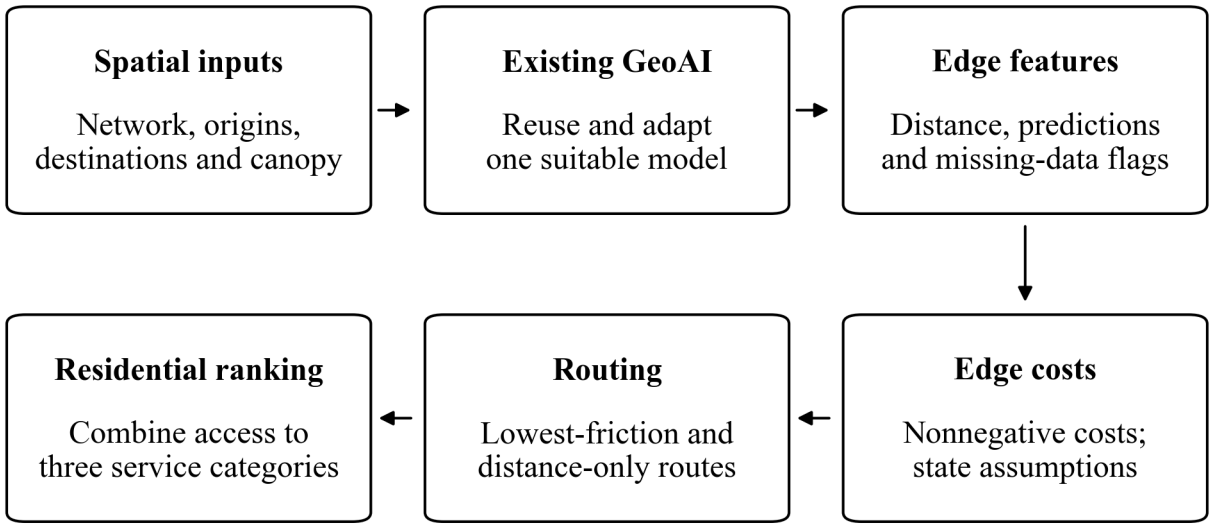
5 Reuse or Adapt One GeoAI Component

Choose one feasible approach: integrate an existing environmental predictor with compatible public inputs, or adapt a released shade–distance route-choice workflow. New model architecture or training from scratch is not required.

- Check that the chosen method meets the hackathon's GeoAI requirement. Record model/version, license, input features, predicted quantity, original training geography and local adaptation.
- For the Singapore study, inspect the public data and reproduce its released analysis before modifying it. Keep participants grouped when fitting and evaluating. Behavioral calibration is not automatically deep learning.
- Use XGBoost or LightGBM only if a suitable existing workflow and meaningful observed labels are available. Install only the selected model's dependencies.
- Keep route-level predictions separate from additive edge costs. Identify manually specified penalties as assumptions; do not claim learned preference weights for unobserved factors.

Save: model adapter or notebook and MODEL_CARD.md. **Done when** the model runs, outputs are checked and reuse is explicit. A precomputed ML canopy product is an input, not proof of a newly implemented model; record any unresolved GeoAI gap.

How the Components Connect



6 Calculate Routes and Residential Rankings

- Define nonnegative additive edge costs with explicit units. Separate model-derived terms from scenario assumptions. Applying sunlight preferences to canopy requires a disclosed transfer assumption.
- Use NetworkX Dijkstra-based routing for distance-only and environmental-friction scenarios. Find the lowest-cost reachable destination in each of the three categories for each origin.

- Rank origins by an equal-weight sum of category costs. Flag missing or unreachable categories; do not present incomplete origins as fully accessible. Destinations need not occur inside every ZIP.

Tools: NetworkX, OSMnx, pandas, GeoPandas. **References:** Street Shadow, CoolWalks and Walk Score’s amenity framing. **Save:** routes.gpkg and residential_rankings.csv with origin/ZIP, destination IDs, distances, category costs, total cost and reachability.

7 Evaluate and Build the Demonstration

- Audit at least one route per ZIP and one major crossing. Inspect connectivity, access and suspicious jumps.
- Compare a baseline/alternative route pair using distance, canopy proxy and modeled cost. Test two plausible penalty settings and record ranking changes. Report the result even if routes do not improve.
- Evaluate the reused model separately. Performance on observations outside Plano does not establish accuracy for Plano pedestrians.
- Build one Folium map with boundaries, origins, destinations, ranked-origin popups and route alternatives. Precompute results; add Streamlit only if essential and time remains.

Tools: Folium, pandas, matplotlib and model-specific evaluation tools. **References:** LA sidewalk heat study for research design; City Health Dashboard for communication. **Save:** empath_demo.html, evaluation.csv and LIMITATIONS.md. **Done when** a viewer can explain a ranking; prepare screenshots because basemap tiles may need internet.

8 Package and Present

- Update README.md with installation, interpreter/kernel, execution order, download instructions, attribution and tested model/library versions. Verify the demo from a fresh session.
- Prepare five-minute slides: problem and user; data and scope; graph and GeoAI; demonstration and finding; limitations and next steps. Save a backup recording or screenshots.
- Hand off sources/coverage, clean-data provenance, graph/features, model adapter/card, routes/rankings, evaluation, demo, code and slides. Reuse generated outputs rather than write a separate long report.

Completion check: the demo loads reproducibly, recommendations are explainable, gaps are disclosed and the presentation fits five minutes. Environmental accessibility does not certify safety or medical suitability.

Working Rules

- Use 15 hours for the core workflow and up to 3 additional hours for blockers or polish. Reduce sampled origins or optional layers before expanding the interface or model.
- Python is the reproducible pipeline: VS Code, Jupyter and a browser. ArcGIS Pro is optional for a teammate’s review; exchange GeoPackage/GeoJSON files.
- Optional teammate: own either network/data QA or model evaluation, with a specific file and documented check to return.
- Future expansion: local preference validation, detailed crossing/traffic data, time-dependent heat/shade and point-to-point lowest-friction navigation.

Source and Reference Links

Census ZCTA boundaries — study scope

Plano sidewalks — supporting polygon inventory

Plano trail centerlines — walking connections to verify

Plano address points — candidate origins

Plano parcels — supporting geometry

Texas 2021 canopy — 10 m raster; download size needs planning

Tree Equity Score — neighborhood context

Singapore behavioral study — shade–distance observations

Singapore data and analysis notebook — reuse and inspect

Street Shadow — inspect modules/graphs.py; manually parameterized shade weights

CoolWalks — routing methodology; Julia implementation

LA sidewalk heat study — residential heat-accessibility framework

UNA — origin/destination architecture; optional reference

Walk Score — amenity-access framing

City Health Dashboard — contextual visualization; its walkability uses Walk Score

OSMnx — network and destination query documentation

NetworkX — shortest-path documentation

Rasterio — windowed reading; requires a readable raster, not only a ZIP URL

Folium — interactive-map documentation

Detailed source notes: docs/RELATED_PROJECTS.md and docs/resources.md. Core libraries: GeoPandas, Shapely, pandas, requests, NumPy, OSMnx, NetworkX, Rasterio, Folium and matplotlib. Add only the selected model’s extra dependencies.